

RadiologyAI

Low Level Design Document

API Endpoint Specifications · v2.0

V G Masilamani(DA25S005)

Project: Chest X-Ray Disease Classifier

Model: EfficientNetB0 (PyTorch 2.3.0)

Backend: FastAPI 0.111.0

Total Endpoints: 12

Classes: Normal · Pneumonia · COVID-19

Contents

1	Overview	2
2	System Endpoints	2
2.1	GET /health — Liveness Probe	2
2.2	GET /ready — Readiness Probe	2
2.3	GET /api/v1/classes — Disease Classes	3
3	Inference Endpoints	3
3.1	POST /api/v1/predict — X-Ray Classification (Core Endpoint)	3
3.1.1	Request Schema	3
3.1.2	Response Schema (200 OK)	4
4	Feedback Endpoints	4
4.1	POST /api/v1/feedback — Submit Radiologist Feedback	4
4.1.1	Request Schema	5
4.1.2	Response Schema (200 OK)	5
4.2	GET /api/v1/feedback/stats — Feedback Statistics	5
4.3	GET /api/v1/feedback/history — Scan History	5
5	Drift Detection Endpoints	5
5.1	GET /api/v1/drift/summary — Drift Summary	6
5.2	GET /api/v1/drift/report/{cls_name} — Evidently HTML Report	6
6	MLflow & Report Endpoints	6
6.1	GET /api/v1/mlflow/runs — Experiment Runs	6
6.2	POST /api/v1/report/pdf — Clinical PDF Report	6
7	Monitoring Endpoint	7
7.1	GET /metrics — Prometheus Metrics	7
8	Acceptance Criteria	8

Overview

This document defines the complete API interface specification for the RadiologyAI back-end service. All 12 endpoints are implemented using FastAPI with Pydantic validation schemas. The React.js frontend communicates exclusively via these REST endpoints — fully decoupled, with the API base URL configured via the `REACT_APP_API_URL` environment variable.

primaryblue	primaryblue
Base URL	<code>http://localhost:8005</code>
Framework	FastAPI 0.111.0 + Python 3.10
Content-Type	<code>application/json</code> (except file upload: <code>multipart/form-data</code>)
API Prefix	<code>/api/v1</code>
Swagger UI	<code>http://localhost:8005/docs</code>
Config	All values loaded from <code>.env</code> via Pydantic Settings — zero hardcoding

System Endpoints

GET /health — Liveness Probe

Checks if the API process is alive. Used by Docker health checks and orchestrators.

primaryblue	primaryblue
Method	GET
URL	<code>/health</code>
Auth	None
Response 200	<code>{ status, model_loaded, version, model_name, framework, image_size, class_count }</code>

Listing 1: Example Response

```
{
  "status": "ok",
  "model_loaded": true,
  "version": "1.0.0",
  "model_name": "EfficientNetB0",
  "framework": "PyTorch 2.3.0",
  "image_size": 224,
  "class_count": 3
}
```

GET /ready — Readiness Probe

Checks if the model is loaded and API is ready to serve predictions.

primaryblue	primaryblue
Method	GET
URL	/ready
Response (ready)	{ "status": "ready", "model_loaded": true }
Response (not ready)	{ "status": "not ready", "model_loaded": false }

GET /api/v1/classes — Disease Classes

primaryblue	primaryblue
Method	GET
URL	/api/v1/classes
Response 200	{ "classes": ["Normal", "Pneumonia", "COVID19"], "total": 3 }

Inference Endpoints

POST /api/v1/predict — X-Ray Classification (Core Endpoint)

Accepts a chest X-ray image, runs EfficientNetB0 inference, generates Grad-CAM heatmap, and returns structured diagnosis result.

primaryblue	primaryblue
Method	POST
URL	/api/v1/predict
Content-Type	multipart/form-data
Accepted formats	JPEG (.jpg, .jpeg), PNG (.png)
Max file size	10 MB
Error 422	Invalid file type or file too large
Error 503	Model not loaded — retry after 30s
Error 500	Model inference failure

Request Schema

primaryblue	primaryblue	primaryblue	primaryblue
file	UploadFile	Yes	Chest X-ray image (JPEG or PNG, max 10MB)

Response Schema (200 OK)

primaryblue	primaryblue	primaryblue
predicted_class	string	Predicted disease: Normal / Pneumonia / COVID19
confidence	float	Confidence score [0.0 – 1.0]
risk_level	string	Low (Normal) / High (Pneumonia, COVID19)
all_probabilities	array	[{ class_name: string, confidence: float }] for all 3 classes
gradcam_base64	string—null	Base64-encoded PNG Grad-CAM heatmap overlay
inference_time_ms	float	Total model inference time in milliseconds
mlflow_run_id	string	MLflow run ID for experiment traceability
disclaimer	string	Mandatory clinical disclaimer text

Listing 2: Example Response

```
{
  "predicted_class": "Pneumonia",
  "confidence": 0.8926,
  "risk_level": "High",
  "all_probabilities": [
    {"class_name": "Normal", "confidence": 0.0351},
    {"class_name": "Pneumonia", "confidence": 0.8926},
    {"class_name": "COVID19", "confidence": 0.0723}
  ],
  "gradcam_base64": "iVBORw0KGgo...",
  "inference_time_ms": 142.3,
  "mlflow_run_id": "78c9be21",
  "disclaimer": "AI-assisted preliminary assessment only..."
}
```

Feedback Endpoints

POST /api/v1/feedback — Submit Radiologist Feedback

Collects radiologist confirmation or correction. Updates Prometheus `radiologyai_model_accuracy` gauge in real time. Persists to `data/feedback.json` for future retraining. Gauge is restored from file on app startup — survives Docker restarts.

Request Schema

primaryblue	primaryblue	primaryblue	primaryblue
prediction_id	string	Yes	Unique prediction identifier
predicted_class	string	Yes	Class predicted by the model
radiologist_confirmed	boolean	Yes	True if AI prediction is correct
correct_class	string	No	Actual class if prediction was wrong
comments	string	No	Optional radiologist clinical notes

Response Schema (200 OK)

primaryblue	primaryblue	primaryblue
status	string	'success' or 'error'
message	string	Confirmation with updated accuracy, e.g. <i>"Feedback recorded. Accuracy: 88.9% (9 samples)"</i>

GET /api/v1/feedback/stats — Feedback Statistics

Returns aggregated accuracy from all radiologist feedback. Called by Airflow DAG to decide retraining.

primaryblue	primaryblue	primaryblue
total_feedback	integer	Total feedback submissions
correct	integer	Correct predictions confirmed
incorrect	integer	Incorrect predictions reported
overall_accuracy	float	Rolling accuracy [0.0–1.0]
per_class	object	{ Normal: { total, correct, accuracy }, Pneumonia: {...}, COVID19: {...} }
retrain_needed	boolean	True if accuracy $\leq$ 0.80 AND total feedback $\geq$ 10

GET /api/v1/feedback/history — Scan History

Returns all scan history for the Patient History page. Sorted by timestamp descending.

primaryblue	primaryblue
history	Array of entries: timestamp, prediction_id, predicted_class, confidence, radiologist_confirmed, correct_class, comments
total	Total number of history entries

Drift Detection Endpoints

GET /api/v1/drift/summary — Drift Summary

Returns Evidently AI drift detection results for all 3 classes.

primaryblue	primaryblue
status	'drift_detected' or 'no_drift'
any_drift	Boolean — true if any class drift score $\geq$ 0.3
results	{ Normal: { drift_detected, drift_score, report_path }, Pneumonia: {...}, COVID19: {...} }

GET /api/v1/drift/report/{cls_name} — Evidently HTML Report

Returns full Evidently AI HTML report for embedding in the Drift page iframe.

primaryblue	primaryblue
Method	GET
URL	/api/v1/drift/report/{cls_name}
Path param	cls_name: Normal — Pneumonia — COVID19
Response 200	HTML content — full Evidently drift report
Response 404	Report not found — run drift_detection.py first

MLflow & Report Endpoints

GET /api/v1/mlflow/runs — Experiment Runs

Proxies MLflow tracking server. Returns all runs for the Model Comparison page.

primaryblue	primaryblue	primaryblue
runs	array	Array of run objects
run_id	string	Short run ID (8 chars)
status	string	FINISHED / FAILED / RUNNING
params	object	batch_size, epochs_phase1, epochs_phase2, lr_phase1, model, git_commit
metrics	object	val_f1, val_acc, test_f1, test_acc, test_loss, macro_auc
total	integer	Total number of runs

POST /api/v1/report/pdf — Clinical PDF Report

Generates downloadable PDF using ReportLab. Includes diagnosis, confidence, class probabilities, Grad-CAM heatmap, patient details, and mandatory disclaimer.

primaryblue	primaryblue	primaryblue
predicted_class	string	Disease classification result
confidence	float	Confidence score [0.0–1.0]
risk_level	string	Low / High
all_probabilities	array	Class probability array
gradcam_base64	string	Base64 Grad-CAM — embedded in PDF
inference_time_ms	float	Inference latency
patient_id	string	Patient identifier
age	string	Patient age

primaryblue	primaryblue
Content-Type	application/pdf
Content-Disposition	attachment; filename=RadiologyAI_Report- <code>{patient_id}</code> - <code>{time}</code> .pdf
Error 500	reportlab not installed or generation failed

Monitoring Endpoint

GET /metrics — Prometheus Metrics

Exposes Prometheus-format metrics scraped every 15 seconds.

primaryblue	primaryblue	primaryblue	primaryblue
http_requests_total	Counter	method, handler, status	Total HTTP re-requests
http_request_duration_seconds	Histogram	handler	Request la-tency — p50/p95/p99
radiologyai_model_accuracy	Gauge	—	Rolling accu-racy from feed-back. Re-stored from file on startup.
radiologyai_feedback_total	Counter	result	Feedback by correc-t/in-correct
radiologyai_feedback_by_class	Counter	predicted_class, result	Feedback by class and result

Acceptance Criteria

primaryblue	primaryblue	primaryblue
Classification Accuracy	≥ 92%	MLflow test_acc metric
Macro F1 Score	≥ 0.92	MLflow test_f1 metric
Macro AUC-ROC	≥ 0.95	MLflow macro_auc metric
Inference Latency (GPU)	≤ 200ms	inference_time_ms in predict re-sponse
API Error Rate	≤ 5%	Prometheus error rate gauge
Model Accuracy (feedback)	≥ 80%	radiologyai_model_accuracy gauge
Drift Detection	Functional	GET /api/v1/drift/summary
PDF Report	Downloadable	POST /api/v1/report/pdf
Test Pass Rate	100%	pytest -html report